“请阅读我的 SillyTavern 开发协议。我现在需要你在这个框架下，帮我增加一个 [这里填你想要的新功能] 功能。请确保所有 JS 逻辑符合局部刷新和拦截注入规范，并保持代码压缩，防止被编辑器吞掉。”

🤖 AI 开发者协议（更新版）
目标：为 SillyTavern / SexyAI 环境开发「逻辑拦截」「本地持久化」「响应式布局」HTML Block，结合本次调试（2026-03-14）实测结论，修订如下细则：

1 脚本生存与逻辑执行（Logic & Security）

* 极简主义：所有行为逻辑优先写入 `onclick` 或极短 `<script>`（单行/单函数），避免冗余命名；变量名尽量极短（例如 `t`,`o`,`d`）。
* 局部重绘：禁止 `location.reload()`。任何更新用数据驱动局部 DOM（`innerHTML="` 后重建或直接修改节点文本）。
* 延迟回显：涉及 DOM 查找（例如发送输入框）必须包含后备重试：`setTimeout(fn,500)`（或一轮短重试），以兼容渲染延迟。
* 输入同步：写入输入框后**必须**触发平台事件：

  * 对 textarea：`t.dispatchEvent(new Event('input',{bubbles:true}))`
  * 对 contenteditable：`t.dispatchEvent(new InputEvent('input',{bubbles:true}))`
    并在必要时执行 `t.focus()` 以确保 React/平台监听器收到变更。
* 拦截机制：若实现 Enter 拦截，监听 `keydown` 时使用捕获阶段（`addEventListener('keydown',fn,true)`），当外层控制 checkbox（id="t"）为 checked 则 `e.preventDefault()`。
* 双击发送策略：注入文字时，拦截逻辑应包含是否已存在的检测（可配置）。若允许重复则跳过检测；若不允许重复，使用 `s.indexOf(o)===-1` 检测并加入。注入后调用上述 dispatch。

2 UI / CSS（Layout）

* Flex 对抗：最外层容器强制 `display:block!important; clear:both;` 以防平台 flex 覆盖。
* 栅格与响应：按钮容器使用 `display:flex; flex-wrap:wrap; gap:6px;`，按钮最小宽度用 `flex:1 1 32%` 或 `48%`（两列/三列可切换）以保证移动端自适应。
* 原子化 CSS：使用独特前缀类名（如 `bbwrap`,`bbbtn`,`bbdialog`）避免被主题覆盖。只注入一次 `<style>`（静态 `<style>` 比动态 `createElement('style')` 更稳定）。设置 `z-index:999999`。
* 视觉保留：按键 active/hover 样式可通过 `.bbbtn.active` 控制（颜色/边框/阴影），不要用会触发平台审查的动画库。

3 结构与交互（Structure）

* 折叠：需要折叠交互时使用 `<details><summary>`（name 属性可做互斥测试）。
* 数据回显与小格子：预览区（dialog）与发送区同步逻辑必须一致；删除采用 `split(o).join('')` 或 `replace` 的组合以兼容换行/空格情况。
* hover/说明：若保留 hover 说明，改用 `onmouseenter` 向下追加（`if(d.innerText.indexOf(t)==-1)

...`)，并提供清空按键以避免累积过长。
* 单/多选策略（已定）：默认**不同项可共存**（$27 $28 $29），**同项不重复**（禁止 $27 $27 $27）；此行为应可配置（注释标记“allowDuplicates:true/false”）。

4 数据持久化与同步（Storage）

* 延迟回显：读取 localStorage 或初始化状态时封装 `setTimeout(fn,500)`，在回调内查找挂载点并渲染状态。
* 状态开关：建议使用隐藏的 `input[type="checkbox"]` 作为全局开关（`.tp` CSS 切换），JS 只操作 `.checked` 值。
* 持久化：若需要记忆按钮状态，保存极简键值到 `localStorage`（键名短），并在 `setTimeout(...,500)` 内恢复。

5 实战模板（极短 onclick 模板，务必保持压缩，并按需替换 $23..$30）

* 说明：按钮显示使用 `$23/$24/$25/$26`；写入内容使用 `$27/$28/$29/$30`。以下为压缩模板（单行）示例——直接放入 `onclick`，满足上面所有要点（延迟、选择器优先级、contenteditable 支持、dispatch、split 删除、不允许同项重复）：

```
(function(b){var o='$27',d=document.getElementById('dialog_$1'),S=['.chat .chat-bottom .uni-textarea .chat-input-scope textarea','#send_textarea textarea','#send_textarea','textarea','[contenteditable=true]'],t;for(var i=0;i<S.length;i++){t=document.querySelector(S[i]);if(t)break;}if(!t||!d){setTimeout(function(){for(var j=0;j<S.length;j++){t=document.querySelector(S[j]);if(t)break;}if(!t||!d)return;_do(t,o,d,b);},500);return;} _do(t,o,d,b);function _do(t,o,d,b){var s=(t.isContentEditable?t.innerText:t.value)||'';if(b.classList.contains('active')){b.classList.remove('active');if(t.isContentEditable){t.innerText=s.split(o).join('').trim();t.dispatchEvent(new InputEvent('input',{bubbles:true}));}else{t.value=s.split(o).join('').trim();t.dispatchEvent(new Event('input',{bubbles:true}));}d.innerText=(d.innerText||'').split(o).join('').trim();}else{if(s.indexOf(o)===-1){b.classList.add('active');if(t.isContentEditable){t.innerText=(s?(s+' '):'')+o;t.dispatchEvent(new InputEvent('input',{bubbles:true}));}else{t.value=(s?(s+' '):'')+o;t.dispatchEvent(new Event('input',{bubbles:true}));}d.innerText=(d.innerText?(d.innerText+'\n'):'')+o;}}})
```

6 额外约定（从本次调试得出）

* 优先用静态 `<style>` 注入一次，后续仅输出 HTML 结构。
* 选择器列表需在项目中统一（把上面 S 数组写入文档），任何新增环境只需在数组头添加新的 selector 即可。
* 所有事件触发后必须 `dispatchEvent(...{bubbles:true})` 以保证平台同步。
* 若出现无法写入（特殊 React 编辑器），请提供输入框 outerHTML，按 DOM 精确写 selector。